

Plegamiento de proteínas usando dinámica molecular

José Ramón Gutiérrez Trujillo

May 8, 2026

1 Introducción

Este trabajo intenta replicar computacionalmente el proceso de plegamiento de una proteína usando un modelo coarse-grained inspirado en el trabajo de Honeycutt y Thirumalai, *The Nature of Folded States of Globular Proteins*. El objetivo es reconstruir el modelo, ejecutar simulaciones de dinámica molecular y analizar observables estructurales y termodinámicos que permitan identificar el régimen de plegamiento, la compactación de la cadena y la posible transición conformacional.

2 Objetivos

2.1 Objetivo general

Reproducir y analizar el plegamiento de una proteína modelo mediante dinámica molecular con un heteropolímero de grano grueso, comparando los observables obtenidos con las ideas de metaestabilidad descritas en el artículo *The Nature of Folded States of Globular Proteins*.

2.2 Objetivos específicos

- Recrear la topología inicial de la cadena heteropolimérica.
- Definir los potenciales enlazantes y no enlazantes del modelo.
- Ejecutar el protocolo de simulación con enfriamiento mediante dinámica de Langevin.
- Calcular energía potencial, calor específico y radio de giro como funciones de la temperatura.
- Interpretar los resultados en términos de compactación y transición estructural.

3 Marco teórico

3.1 Plegamiento de proteínas y paisaje de energía

El plegamiento de proteínas es uno de los problemas más complejos e importantes para la biología y la biofísica. Para que una proteína funcione correctamente, debe plegarse de una forma muy precisa, proceso que, al parecer, está codificado en su secuencia lineal de aminoácidos. La hipótesis termodinámica postula que la estructura nativa corresponde al mínimo global de energía libre. Lo interesante viene por el hecho de que el tiempo requerido para que la cadena explore todo el espacio de forma aleatoria es, para términos prácticos, infinito (paradoja de Levinthal). Y se sabe que las proteínas se suelen plegar en cuestión de minutos o incluso segundos.

Estados posibles

- **Desplegado:** cadena extendida e inestable, con alta energía libre y exposición máxima de residuos hidrofóbicos al solvente, como en condiciones desnaturalizadas.
- **Colapsado:** estado compacto pero desordenado (glóbulo fundido), con residuos hidrofóbicos agrupados para reducir contactos con el agua; actúa como intermediario hacia el plegado.
- **Plegado (nativo):** estructura ordenada y estable, con interacciones específicas que minimizan la energía libre.

Paisaje de energía

El paisaje de energía se podría simplificar como un mapa de valles y montañas (como el que se ve en la imagen #1), y es lo que define la estabilidad del sistema. Los valles son las zonas de mínimos de energía, donde el sistema es más estable; el valle más profundo es el mínimo global. En el contexto del plegamiento de proteínas, es ahí donde existe la estructura nativa. Los mínimos locales son donde existen los estados colapsados.

Las barreras de energía son las cimas que separan los valles. Para cruzar una barrera, el sistema requiere de una energía de activación térmica. Cuando la barrera es muy alta, el sistema no puede cambiar a otro estado de energía menor y es entonces cuando se queda atrapado en plegamientos incorrectos.

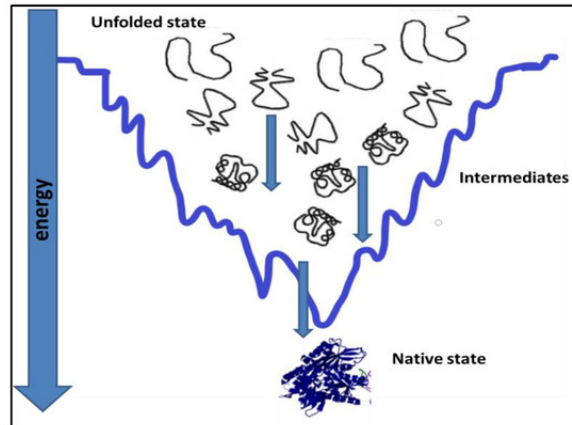


Figura 1. Representación esquemática del paisaje de energía durante el plegamiento de proteínas, donde se distinguen el estado nativo, los mínimos locales y las barreras energéticas asociadas al proceso de plegamiento y mal plegamiento (Ajmal, 2023).

3.2 Hipótesis de metaestabilidad

En el paper, Honeycutt y Thirumalai proponen la hipótesis de metaestabilidad, la cual, comentan, puede verse como una generalización de la hipótesis del control cinético, que establece lo siguiente: existen varias formas de conformaciones plegadas de una cadena polipeptídica, todas ellas con características estructurales muy similares, pero que difieren entre sí en cuanto a su energía.

Por otro lado, la hipótesis de metaestabilidad dice lo siguiente: las características estructurales medidas (o calculadas) se corresponden con las propiedades estadísticas obtenidas al restringir las configuraciones a un mínimo determinado. Además, es esperado que estas propiedades estructurales sean similares entre los diferentes mínimos.

El segundo aspecto importante de la hipótesis es el rol de las condiciones iniciales de plegamiento al direccionar la cadena desnaturalizada hacia uno de los mínimos correspondientes a un estado plegado.

3.3 Modelo de Honeycutt–Thirumalai

Se usa un modelo coarse-grained de una cadena heteropolimérica con tres tipos de residuos, identificados como B, L y N, junto con interacciones locales y de largo alcance que favorecen la formación de una estructura compacta.

Honeycutt y Thirumalai proponen un modelo coarse-grained de una cadena heteropolimérica para comprobar su hipótesis de metaestabilidad en el plegamiento de proteínas. El modelo de coarse-grained permite observar cosas que serían computacionalmente prohibitivas usando modelos de resolución atómica.

3.3.1 Los tres tipos de residuos (B, L, N)

Proponen un modelo de una cadena de 46 unidades compuesta por tres tipos de esferas que representan aminoácidos:

- **B (Hidrofóbicos):** Se atraen entre sí por medio de un potencial de Lennard-Jones. Esta es la interacción principal del modelo, ya que es la causante de que la proteína colapse en la estructura nativa.
- **L (Hidrofílicos):** Estas esferas son repulsivas cuando interactúan con esferas L o B. Son las encargadas de oponer un poco de resistencia a que la cadena colapse.
- **N (Neutros):** Interactúan únicamente a corto alcance y son las encargadas de los giros en el plegamiento. Pero interactúan tanto con Bs, Ls y Ns.

3.3.2 Interacciones de largo alcance

Las interacciones de largo alcance ocurren entre esferas que pueden estar muy separadas en la secuencia lineal de la cadena.

Su función se da mediante la atracción de las esferas B-B, que compensa la rigidez de la cadena y es lo que propicia un estado de baja energía. También es importante mencionar que, si la atracción es débil, la cadena queda desplegada; y, si es demasiado fuerte, lleva a estados colapsados.

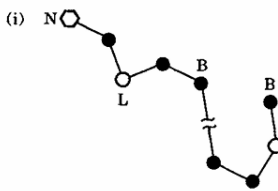
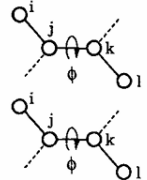
INTERACTION TYPE	FUNCTIONAL FORM
(i)  Long Range Interaction	$V_{BB}(r) = 4\epsilon_h [(\sigma/r)^{12} - (\sigma/r)^6]$ $V_{L\alpha}(r) = 4\epsilon_L [(\sigma/r)^{12} + (\sigma/r)^6]$ $\epsilon_L = 2/3 \epsilon_h \quad (\alpha = L \text{ or } B)$ $V_{N\alpha}(r) = 4\epsilon_h (\sigma/r)^{12}$
(ii)  Bond Angle Potential	$V(\theta) = 1/2 k_\theta (\theta - \theta_0)^2$ $k_\theta = 20\epsilon_h / (\text{rad})^2; \theta_0 = 105^\circ$
(iii)  Dihedral Angle Potential	$(i, j, k \neq N)$ $V(\phi) = A (1 + \cos \phi) + B (1 + \cos 3\phi)$ $A = B = 1.2\epsilon_h$ $(i, j = N; k, l \neq N) ; (i, j, k = N)$ $V(\phi) = B (1 + \cos 3\phi)$ $B = 0.2\epsilon_h$

Figura 2. Esquema de las interacciones consideradas en el modelo de Honeycutt-Thirumalai. Se ilustran las interacciones de largo alcance entre residuos no enlazados y los potenciales locales que controlan los ángulos de enlace y los ángulos diedros, los cuales determinan la rigidez de la cadena y la formación de giros durante el plegamiento (Honeycutt & Thirumalai, 1992).

3.3.3 Interacciones locales (Potenciales angulares)

- **Potencial de Ángulo de Enlace:** Mantiene los ángulos entre tres esferas sucesivas cerca de 105° mediante un potencial armónico.
- **Potencial de Ángulo Diedro:** Controla la rotación alrededor de los enlaces. Donde hay Ns, este potencial se debilita para facilitar que la cadena se doble sobre sí misma.

4 Metodología computacional

4.1 Protocolo de simulación

El código que se implementó se divide en tres etapas:

1. Construcción de la topología inicial (un único script).

2. Ejecución del protocolo de simulación (un único script).
3. Análisis de los resultados (dividido en tres scripts) para calcular el radio de giro, calor específico y energía.

Para su interpretación, estos se comparten en las siguientes secciones con comentarios muy descriptivos.

4.1.1 build_topology.py

Este es el script usado para generar la topología inicial.

```
import gsd
import gsd.hoomd
import numpy as np

def build_ht_model(filename="ht_model_initial.gsd"):
    """
    Construye la topología inicial para el modelo de proteína de
    grano grueso
    de Honeycutt y Thirumalai. Genera una cadena lineal completamente
    desplegada para
    evitar fuerzas infinitas (choques estéricos) al inicio de la
    simulación.
    """

    # =====
    # 1. DEFINICIÓN DE LA SECUENCIA DE AMINOÁCIDOS (RESIDUOS)
    # =====
    # Secuencia exacta del paper (n=46): B9 N3 (LB)4 N3 B9 N3 (LB)5 L
    # B = Hidrofóbicos (buscan agruparse en el núcleo).
    # L = Hidrofílicos (buscan el exterior/solvente).
    # N = Neutros (tienen potenciales más débiles para facilitar los
    giros/loops).
    seq = (
        ['B'] * 9 +
        ['N'] * 3 +
        ['L', 'B'] * 4 +
        ['N'] * 3 +
        ['B'] * 9 +
        ['N'] * 3 +
        ['L', 'B'] * 5 +
        ['L']
    )
    n_particles = len(seq)

    # =====
    # 2. INICIALIZACIÓN DEL ENTORNO DE SIMULACIÓN (FRAME)
    # =====
```

```

frame = gsd.hoomd.Frame()
frame.configuration.step = 0
frame.configuration.dimensions = 3

# Definimos la caja de simulación.
frame.configuration.box = [150.0, 150.0, 150.0, 0, 0, 0]

# =====
# 3. CONFIGURACIÓN DE LAS PARTÍCULAS (LOS RESIDUOS)
# =====
frame.particles.N = n_particles

frame.particles.types = ['B', 'L', 'N']

# HOOMD-blue trabaja internamente con números (typeids), no con
# letras.
type_map = {'B': 0, 'L': 1, 'N': 2}
frame.particles.typeid = [type_map[res] for res in seq]

# Asignamos masa 1.0 a todos los residuos (en unidades reducidas)
frame.particles.mass = np.ones(n_particles, dtype=float)

# Coordenadas iniciales: Colocamos las partículas en una línea
# recta a lo largo del eje X.
positions = np.zeros((n_particles, 3), dtype=np.float32)
positions[:, 0] = np.arange(n_particles) * 1.0 # Espaciados
# exactamente por 1.0 sigma
positions[:, 0] -= np.mean(positions[:, 0]) # Centramos la
# cadena en el origen (0,0,0)
frame.particles.position = positions

# =====
# 4. DEFINICIÓN DE LA COLUMNA VERTEBRAL (ENLACES / BONDS)
# =====
# Unimos la partícula i con la i+1 consecutivamente.
frame.bonds.N = n_particles - 1
frame.bonds.types = ['backbone']
frame.bonds.group = np.array([[i, i+1] for i in range(n_particles
- 1)])

# Solo hay un tipo de enlace (typeid = 0), todos con longitud
# objetivo de 1.0
frame.bonds.typeid = np.zeros(n_particles - 1, dtype=int)

# =====
# 5. DEFINICIÓN DE LOS ÁNGULOS DE ENLACE (ANGLES)
# =====
# Definimos el ángulo formado por tres partículas consecutivas: i
# , i+1, i+2
frame.angles.N = n_particles - 2
frame.angles.types = ['harmonic']

```

```

frame.angles.group = np.array([[i, i+1, i+2] for i in range(
    n_particles - 2)])

# Solo hay un tipo de ángulo armónico (typeid = 0), con objetivo
# de 105 grados
frame.angles typeid = np.zeros(n_particles - 2, dtype=int)

# =====
# 6. DEFINICIÓN DE LOS ÁNGULOS DIEDROS Y LA REGLA DE GIRO (BEND)
# =====
# Un diedro define la torsión en 3D de cuatro partículas
# consecutivas: i, i+1, i+2, i+3
frame.dihedrals.N = n_particles - 3

# El paper define dos tipos de diedros:
# 'normal': Fuerza de torsión estándar que favorece la conformaci
# ón trans (estirada).
# 'bend' : Fuerza más débil que permite a la cadena doblarse
# sobre sí misma.
frame.dihedrals.types = ['normal', 'bend']

d_groups = []
d_typeids = []

for i in range(n_particles - 3):
    group = [i, i+1, i+2, i+3]
    d_groups.append(group)

    # Obtenemos los tipos de aminoácidos involucrados en este
    # diedro de 4 partículas
    res_in_dihedral = [seq[j] for j in group]

    # REGLA TERMODINÁMICA DEL PAPER: Si hay al menos un residuo
    # Neutro ('N')
    # en el conjunto de 4 partículas, este diedro se considera
    # una zona de giro ('bend').
    if 'N' in res_in_dihedral:
        d_typeids.append(1) # ID 1 corresponde a 'bend'
    else:
        d_typeids.append(0) # ID 0 corresponde a 'normal'

frame.dihedrals.group = np.array(d_groups)
frame.dihedrals.typeid = np.array(d_typeids)

# =====
# 7. GUARDAR EL ARCHIVO GSD
# =====
# Abrimos (o creamos) el archivo en modo escritura ('w') y
# guardamos el Frame
with gsd.hoomd.open(name=filename, mode='w') as f:
    f.append(frame)

```



```

print(f"Topología inicial construida exitosamente en: {filename}"
)
print(f"La simulación contendrá: {n_particles} partículas, {frame
.bonds.N} enlaces, {frame.angles.N} ángulos y {frame.dihedrals
.N} diedros.")

if __name__ == "__main__":
    build_ht_model()

```

4.1.2 run_sim.py

Este es el script principal, ya que en él se ejecuta todo el protocolo: el calentamiento inicial de la proteína desnaturalizada y, posteriormente, su enfriamiento lento usando una rampa de temperatura. Al final, se corren 500,000 pasos extras para asegurar el equilibrio en la temperatura deseada.

Se agregaron referencias al paper según lo que se va ejecutando en el script, con el fin de mantener la claridad sobre qué se hace y por qué.

```

import hoomd
import hoomd.md
import hoomd.write
import numpy as np
import itertools

def run_ht_folding(init_file, output_traj, temp_init, temp_finish):

    # 1. INICIALIZAR LA SIMULACIÓN
    # El paper utiliza "Noisy Molecular Dynamics" para simular el
    # entorno del solvente.
    # [Ref: Pág. 695, Synopsis; Pág. 700, Simulation Methods]
    device = hoomd.device.CPU()
    sim = hoomd.Simulation(device=device, seed=888)
    sim.create_state_from_gsd(filename=init_file)

    # =====
    # 2. POTENCIALES ENLAZADOS (Basado en el modelo de cadena de
    # carbonos alfa)
    # =====

    # Los enlaces se consideran de longitud fija (r0=1.0) en el paper
    # .
    # [Ref: Pág. 698, Columna 2, Secc. Construction of the Model; Pág
    # . 700, Columna 2]
    # k=1000 simula la rigidez para aproximar la restricción rígida
    # mencionada en la Pág. 699.
    harmonic_bond = hoomd.md.bond.Harmonic()

```

```

harmonic_bond.params['backbone'] = dict(k=1000.0, r0=1.0)

# El ángulo de equilibrio theta_0 es 105° (1.8326 rad).
# [Ref: Pág. 699, Eq. 5 y Fig. 2]
harmonic_angle = hoomd.md.angle.Harmonic()
harmonic_angle.params['harmonic'] = dict(k=20.0, t0=1.8326)

# Potencial diedro: Crucial para estabilizar la forma de barril-
#   beta.
# [Ref: Pág. 698, Columna 2, Secc (c); Pág. 699, Eq. 13]
# Se usan parámetros A=B=1.2 para promover la configuración trans
#   /gauche.
# [Ref: Pág. 699, Columna 2, Dihedral Angle Potential]
opls_dihedral = hoomd.md.dihedral.OPLS()
opls_dihedral.params['normal'] = dict(k1=2.4, k2=0.0, k3=2.4, k4
    =0.0)

# En las regiones de giro (loops/N), el potencial es más débil (A
#   =0, B=0.2)
# para facilitar la formación de curvas.
# [Ref: Pág. 699, Columna 2, Párrafo final; Pág. 700, Fig. 2]
opls_dihedral.params['bend'] = dict(k1=0.0, k2=0.0, k3=0.4, k4
    =0.0)

# =====
# 3. POTENCIALES NO ENLAZADOS (Interacciones de largo alcance)
# =====
nl = hoomd.md.nlist.Cell(buffer=0.4)
# B: Hidrofóbico, L: Hidrofílico, N: Neutro [Ref: Pág. 698,
#   Columna 2]
tipos_particulas = ['B', 'L', 'N']

# A) Interacciones Lennard-Jones: Representan la atracción hidrof
#   óbica.
# [Ref: Pág. 698, Columna 2, Secc (a); Pág. 699, Eq. 2]
# Solo los pares B-B se atraen para inducir el colapso de la
#   proteína.
# [Ref: Pág. 698, Columna 2, "B residues attract other B residues
#   "]
lj = hoomd.md.pair.LJ(nlist=nl)
for t1, t2 in itertools.product(tipos_particulas, repeat=2):
    lj.params[(t1, t2)] = dict(epsilon=0.0, sigma=1.0)
    lj.r_cut[(t1, t2)] = 0.0

lj.params[('B', 'B')] = dict(epsilon=1.0, sigma=1.0)
lj.r_cut[('B', 'B')] = 2.5

# B) Interacciones Repulsivas Tabuladas
# Implementa V_L_alpha y V_N_alpha descritos en el paper.
# [Ref: Pág. 699, Eqs. 3 y 4; Pág. 700, Fig. 2]
table_width = 1000

```

```

r_min_val = 0.5
r_cut_tab = 2.5
r_array = np.linspace(r_min_val, r_cut_tab, table_width, endpoint
                      =False)

# Potencial para residuos L: Puramente repulsivo con epsilon_L =
  2/3 epsilon_h.
# [Ref: Pág. 699, Eq. 3 y Columna 1 "purely repulsive"]
V_L_alpha = 4.0 * (2.0/3.0) * (r_array**-12 + r_array**-6)
F_L_alpha = 4.0 * (2.0/3.0) * (12.0*r_array**-13 + 6.0*r_array
                              **-7)

# Potencial para residuos N: Repulsión de esfera blanda (r^-12).
# [Ref: Pág. 698, Columna 2, "short-range repulsion"; Pág. 699,
  Eq. 4]
V_N_alpha = 4.0 * (r_array**-12)
F_N_alpha = 4.0 * (12.0*r_array**-13)

zero_array = np.zeros(table_width)
table = hoomd.md.pair.Table(nlist=nl)

for t1, t2 in itertools.product(tipos_particulas, repeat=2):
    table.params[(t1, t2)] = dict(r_min=r_min_val, U=zero_array,
                                  F=zero_array)
    table.r_cut[(t1, t2)] = 0.0

table.params[('L', 'L')] = dict(r_min=r_min_val, U=V_L_alpha, F=
    F_L_alpha)
table.params[('L', 'B')] = dict(r_min=r_min_val, U=V_L_alpha, F=
    F_L_alpha)
table.params[('B', 'L')] = dict(r_min=r_min_val, U=V_L_alpha, F=
    F_L_alpha)
table.r_cut[('L', 'L')] = table.r_cut[('L', 'B')] = table.r_cut[(
    'B', 'L')] = r_cut_tab

for t in tipos_particulas:
    table.params[('N', t)] = dict(r_min=r_min_val, U=V_N_alpha, F
    =F_N_alpha)
    table.params[(t, 'N')] = dict(r_min=r_min_val, U=V_N_alpha, F
    =F_N_alpha)
    table.r_cut[('N', t)] = table.r_cut[(t, 'N')] = r_cut_tab

# =====
# 4. INTEGRACIÓN Y TERMOSTATO (Langevin / Stochastic Dynamics)
# =====
# El uso de fricción (gamma) y ruido controla la temperatura físicamente.
# [Ref: Pág. 700, Columna 2, "noisy molecular dynamics"; Pág.
  701, Eq. 11]
integrator = hoomd.md.Integrator(dt=0.005)
integrator.forces.extend([harmonic_bond, harmonic_angle,

```

```

        opls_dihedral, lj, table])

langevin = hoond.md.methods.Langevin(filter=hoond.filter.All(),
    kT=2.0)
for particle_type in tipos_particulas:
    langevin.gamma[particle_type] = 1.0

integrator.methods.append(langevin)
sim.operations.integrator = integrator

# =====
# 5. PROTOCOLO DE PLEGAMIENTO (Simulated Annealing) & LOGGING
# =====

thermo = hoond.md.compute.ThermodynamicQuantities(filter=hoond.
    filter.All())
sim.operations.computes.append(thermo)

logger = hoond.logging.Logger()
logger.add(thermo, quantities=['potential_energy', '
    kinetic_energy'])

gsd_writer = hoond.write.GSD(filename=output_traj,
    trigger=hoond.trigger.Periodic(100),
    mode='wb')

# --- Imprimimos la información del logger ---
gsd_writer.logger = logger
sim.operations.writers.append(gsd_writer)

print("Iniciando fase de desnaturalización (T=2.0)...")
sim.run(50000)

print("Iniciando enfriamiento lento (Simulated Annealing)...")
ramp = hoond.variant.Ramp(A=temp_init, B=temp_finish, t_start=sim
    .timestep, t_ramp=500000)
langevin.kT = ramp

# --- Comenzamos el enfriamiento ---
sim.run(500000) # Esta línea ejecuta únicamente la rampa de
    enfriamiento.

print(f"Iniciando equilibración a T={temp_finish}...")
sim.run(500000) # Corremos otros 500,000 pasos para asegurar el
    equilibrio.

print(f"Simulación terminada. Trayectoria guardada en: {
    output_traj}")

if __name__ == "__main__":
    for i in np.arange(0, 2.05, 0.05):

```

```
run_ht_folding(init_file="ht_model_initial.gsd", output_traj=
    f'folding_t_{i:.2f}.gsd', temp_init = 2.0, temp_finish = i
)
```

5 Observables y análisis de datos

5.1 Energía potencial

En HOOMD-blue, la propiedad `potential_energy` (U) es la suma de todas las contribuciones de fuerza definidas en el sistema. Según la documentación, la energía potencial total que un subconjunto de partículas aporta al sistema se calcula mediante la siguiente expresión:

$$U = U_{net,additional} + \sum_{i \in filter} U_{net,i}$$

Donde:

- U : Es la energía potencial total en unidades de energía.
- $i \in filter$: Representa el conjunto de partículas seleccionadas (en el caso de nuestro sistema, el conjunto es toda la cadena) para el cálculo.
- $U_{net,i}$: Son los términos de energía neta calculados por `hoomd.md.Integrator`.
- Fuerzas: El integrador procesa todas las fuerzas contenidas en el comando `hoomd.md.Integrator.forces` (como fuerzas de enlace, fuerzas de pares como Lennard-Jones, etc.) para derivar estos términos.

Entonces, lo implementado en el script *energy.py* (vease el anexo correspondiente) extrae la energía potencial del archivo `gsd`. Luego, se procede a quedarse únicamente con el último 30% de los frames para asegurar que solo estamos midiendo la energía después de que la rampa de enfriamiento haya terminado. **Esto último, es decir, tomar solo los últimos frames, también aplica para los calculos de radio de giro y el calor específico, cosa que ya no se mencionara en dichas secciones.**

Las líneas importantes son:

```
for frame in traj:
    if frame.log and log_key in frame.log:
        pe = frame.log[log_key]
        # Asegura que sea un valor escalar (HOOMD a veces lo guarda
        # como array)
        if isinstance(pe, (list, np.ndarray)):
            pe = pe[0]
        energies.append(pe)
```

Este es el calculo más sencillo que se implementó, porque en realidad ya lo calculo HOOMD. En este trabajo, simplemente se calculo el promedio de los últimos frames para asegurar el equilibrio después del enfriamiento.

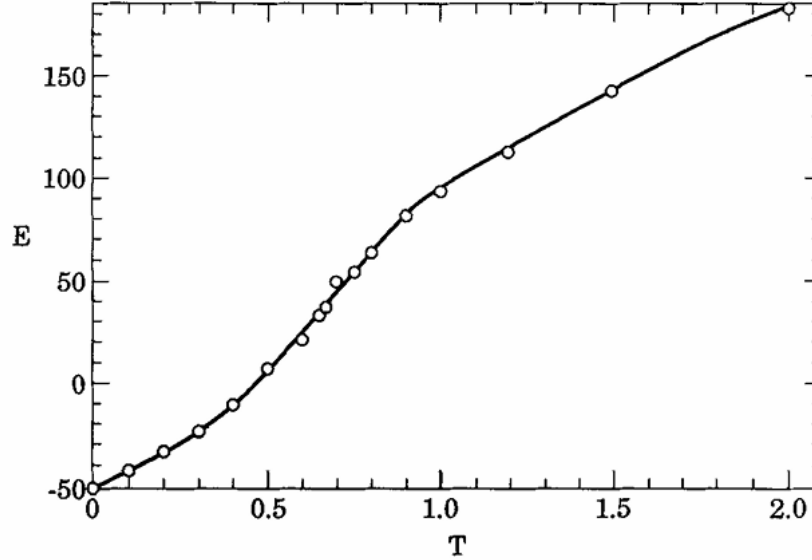
Para cada temperatura T_i , se calcula:

$$\langle U \rangle_{T_i} = \frac{1}{M} \sum_{j=1}^M U_j$$

Donde M es el número de frames en el último 30% de la trayectoria.

Al graficar estos promedios frente a la temperatura, se obtiene la curva de Energía Potencial Promedio vs. Temperatura, que se usa para identificar transiciones de fase (como el punto de plegado de una proteína o la fusión de un sólido). Si la curva presenta un salto, esto indica una transición estructural importante en esa región de temperatura.

Por ejemplo, la gráfica presentada en el paper se ve así:



5.2 Radio de giro

En la física de polímeros y proteínas, el radio de giro se usa para cuantificar el tamaño global y la compactación estructural del sistema. En este caso, solo lo usamos para lo último.

El radio de giro mide la distribución espacial de nuestras esferas alrededor de su centro de masa. Su definición matemática para un sistema de N partículas con las mismas masas es:

$$R_g^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{r}_i - \mathbf{r}_{cm})^2$$

Donde $\mathbf{r}_i$ es la posición de la partícula i y $\mathbf{r}_{cm}$ es la posición del centro de masa del polímero. Estructura Compacta: Si la proteína está plegada (como una bola densa), las partículas están muy cerca del centro de masa, resultando en un R_g pequeño. Estructura Desplegada: Si la cadena está estirada o en forma de ovillo aleatorio (random coil), las partículas se alejan del centro, resultando en un R_g grande.

En el script `radius_of_gyration.py` (ver anexo correspondiente), el radio se calcula con las siguientes líneas de código:

```
# Inicializamos freud
cl_props = freud.cluster.ClusterProperties()

for frame in frames_to_analyze:
    box = frame.configuration.box
    positions = frame.particles.position

    # Asumiendo que todo el sistema es un solo polímero
    cluster_keys = np.zeros(len(positions), dtype=np.int32)
    system = (box, positions)

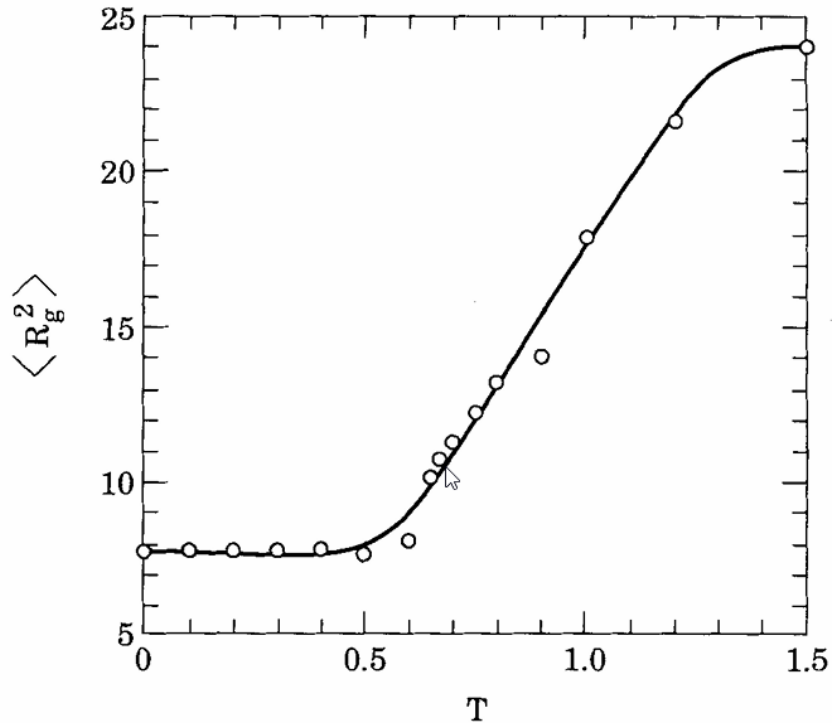
    # Calcular propiedades
    cl_props.compute(system, cluster_idx=cluster_keys)

    # Extraer el radio de giro de este frame
    rg = cl_props.radii_of_gyration[0]

    rg_list.append(rg)
    rg_squared_list.append(rg**2)

# Calcular promedios
avg_rg = np.mean(rg_list)
avg_rg_squared = np.mean(rg_squared_list)
```

Al graficar $\langle R_g^2 \rangle$ contra la temperatura T se obtiene una curva en forma de S que sirve como un termómetro estructural. Por ejemplo, la gráfica presentada en el paper se ve así:



5.3 Calor específico

El calculo del calor específico se implementó en *specific_heat.py* (vease el anexo correspondiente).

La ecuación implementada es

$$C_v = \frac{\langle U^2 \rangle - \langle U \rangle^2}{T^2} + \frac{3}{2}N$$

, la cual se divide en dos contribuciones físicas:

- **Configuracional:** Que sería el término $\frac{\langle U^2 \rangle - \langle U \rangle^2}{T^2}$. Está asociado al hecho de que al darle calor a un sistema, no todo se gasta en aumentar la velocidad de las esferas. Parte de esa energía también se gasta en estirar enlaces, vencer fuerzas de atracción (Lennard-Jones) y cambiar la geometría del sistema. El ensamble canónico establece que la capacidad de un sistema para absorber calor es directamente proporcional a qué tanto fluctua sus energía potencial en equilibrio. Si la energía potencial fluctua mucho ($\langle U^2 \rangle - \langle U \rangle^2$), nos dice que el sistema está oscilando entre varios estados.

- **Contribución Cinética:** Que sería el término $\frac{3}{2}N$. Este término está asociado únicamente al movimiento de las esferas y según el Teorema de Equipartición de la Energía, cada grado de libertad contribuye con $\frac{1}{2}k_B T$ a la energía cinética. La energía cinética total del sistema sería $U_{kin} = \frac{3}{2}Nk_B T$. Pero recordemos que el calor específico es $(\frac{\partial U}{\partial T})$, al derivar nos quedamos con $\frac{3}{2}Nk_B$, o simplemente $\frac{3}{2}N$ en unidades reducidas.

Entonces, lo implementado en el script *specific_heat.py* (vease el anexo correspondiente) extrae la energía potencial del archivo gsd, y el calor específico se calcula de la siguiente manera:

```
# 1. Calcular la Varianza de la Energía Potencial (<U^2> - <U>^2)
# ddof=0 devuelve la varianza, que es la que usa la mecánica
# estadística
energy_variance = np.var(U, ddof=0)

# 2. Calcular el Calor Específico configuracional (Cv_conf = Var(
# U) / (kB * T^2))
# En unidades reducidas, la constante de Boltzmann kB = 1
# Verificación para evitar división por cero si T=0.0
if T == 0.0:
    Cv_configuracional = 0.0
else:
    Cv_configuracional = energy_variance / (T**2)

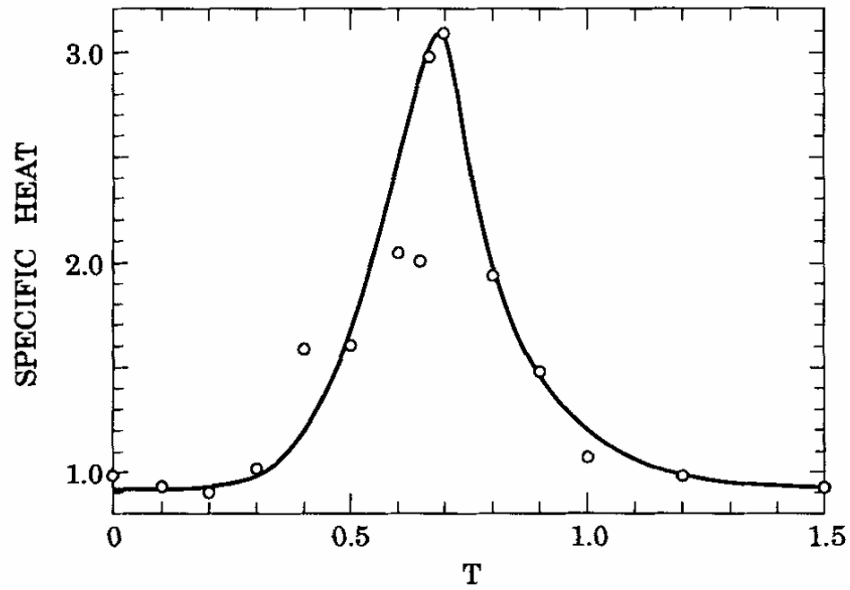
# 3. Calcular la contribución cinética al Calor Específico
# Para un sistema 3D, Cv_cinetico = (3/2) * N * kB
Cv_kinetic = 1.5 * N

# Capacidad Calorífica Total
Cv_total = Cv_configuracional + Cv_kinetic
```

Identificación de la Temperatura de Transición

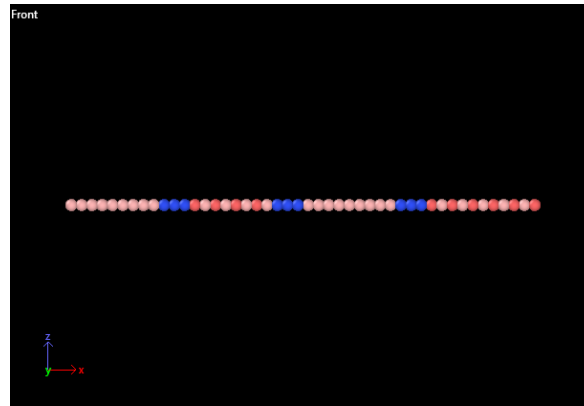
Esta es la forma estándar de encontrar la temperatura de transición (por ejemplo, la temperatura de plegado T_f). En un estado altamente estable (como un sólido o una proteína completamente plegada a baja temperatura), o en un estado completamente desordenado (gas o cadena desplegada a alta temperatura), las fluctuaciones de la energía $\langle U^2 \rangle - \langle U \rangle^2$ son relativamente pequeñas. Pero, exactamente en la temperatura de transición, el sistema oscila constantemente entre el estado ordenado y el desordenado. Esta coexistencia macroscópica provoca fluctuaciones gigantescas en la energía potencial. Al graficar los resultados generados por el script, la temperatura de transición se revelará como un pico agudo y pronunciado en la curva de C_v vs. T .

Por ejemplo, la gráfica presentada en el paper se ve así:



6 Resultados

6.1 Estructura inicial del sistema



La imagen de arriba es una captura del archivo generado por *build_topology.py*. Las esferas azules representan los residuos **N (Neutros)**, las esferas ligeramente naranjas representan los residuos **B (Hidrofóbicos)** y las esferas rojas representan los residuos **L (Hidrofílicos)**. No hay forma visual de representar los ángulos de enlace, los diedros y los enlaces; pero el archivo output del script también los implementa.

6.2 Evolución del plegamiento durante la simulación

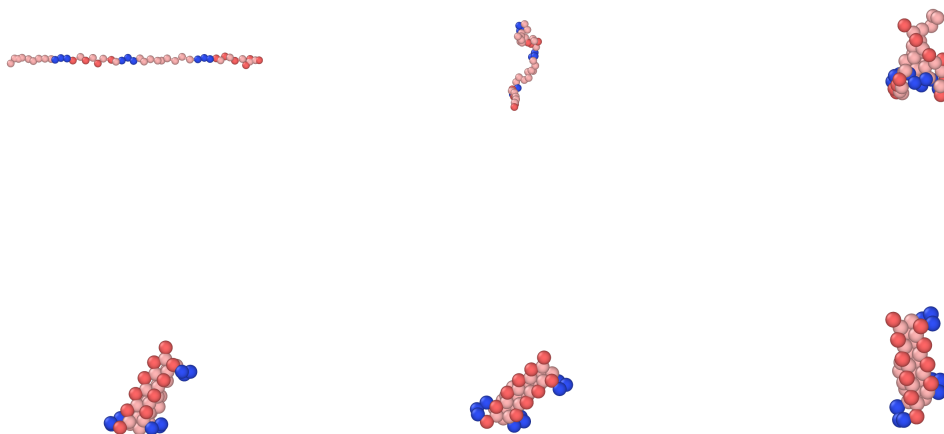
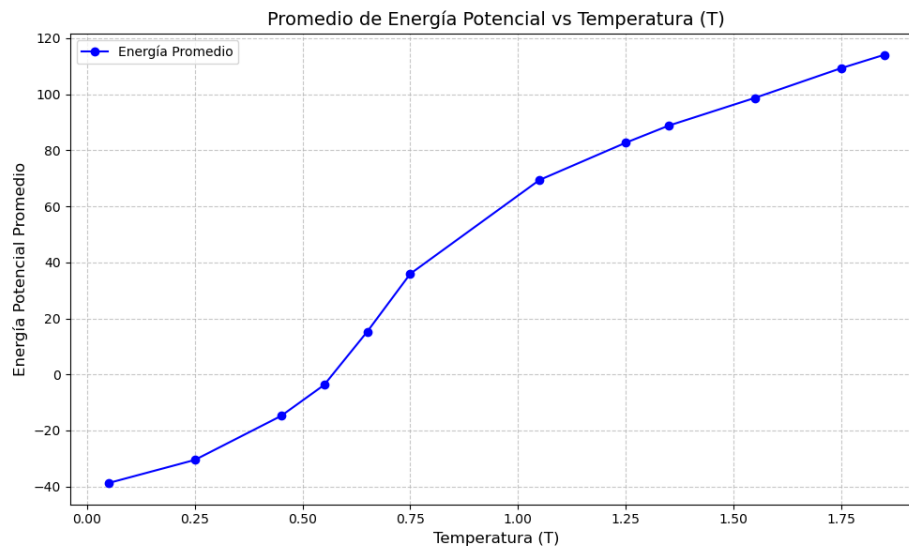


Figura 8. Evolución estructural de la cadena durante la simulación, mostrando configuraciones representativas a 0, 2000, 4000, 6000, 8000 y 10000 pasos.

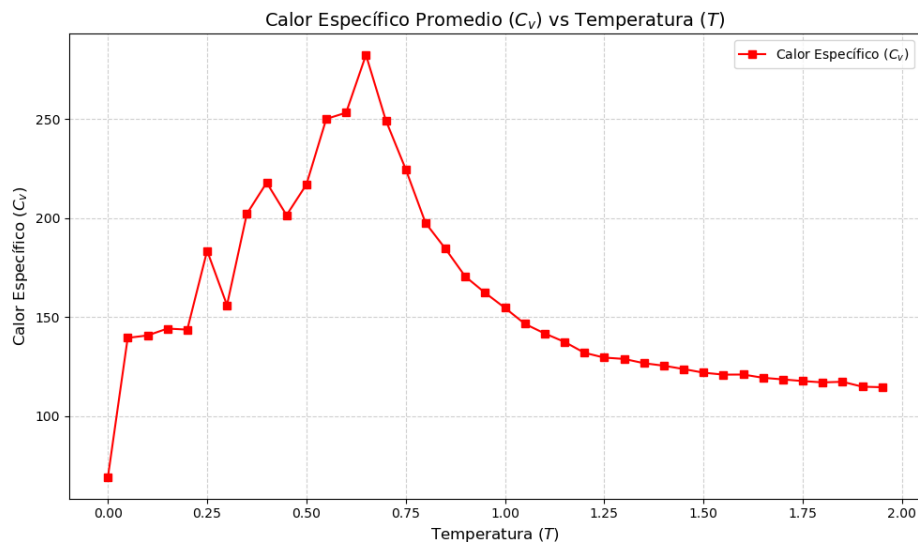
El hecho de que las imágenes correspondientes a 6000, 8000 y 10000 pasos parezcan tan semejantes se debe a que se dejó correr la simulación 500,000 pasos al final para asegurar el equilibrio.

6.3 Energía potencial como función de la temperatura



A bajas temperaturas, por ejemplo $T = 0.05$ o $T = 0.45$, la energía potencial es tan pequeña que incluso tiene valores negativos. Esto indicaría que la cadena está en su estado nativo. Se puede observar una fase de transición entre $T = 0.45$ y $T = 1.00$, pero la verdad no es muy notoria en esta gráfica. Pero sería en esta zona donde la proteína comienza a plegarse. A altas temperaturas, mayores a $T = 1.0$, la energía se ve que aumenta pero de una forma menos pronunciada que en la fase de transición. Esta zona corresponde a la de la proteína desnaturalizada o completamente desplegada.

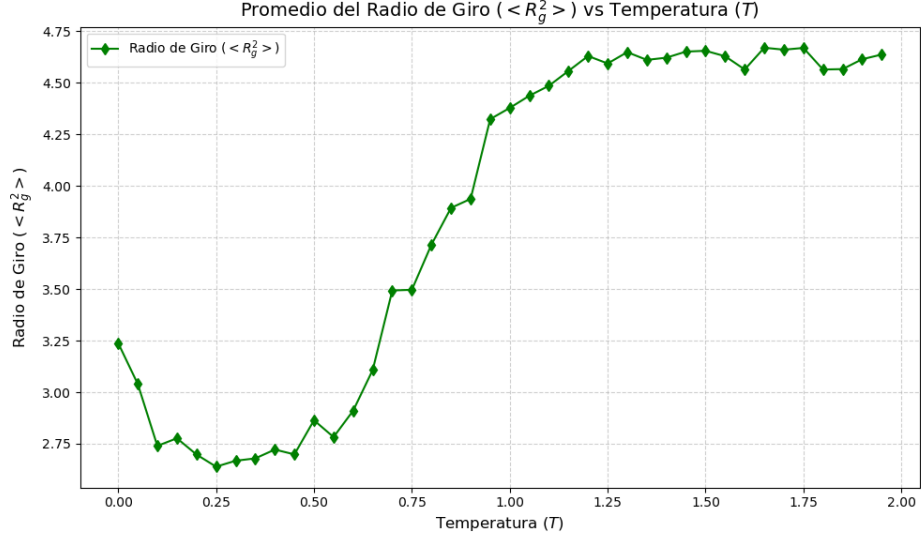
6.4 Calor específico y temperatura de transición



Como se comentó en la sección 5.3, el C_v es una propiedad muy importante para estimar la T_f . En el punto $T = 0.65$, el valor de C_v alcanza su máximo (≈ 282.46). Esto significa que, a esta temperatura, el sistema tiene la mayor variabilidad energética. La proteína está oscilando entre conformaciones plegadas y desplegadas, lo que genera grandes fluctuaciones de energía.

A baja temperatura ($T < 0.4$), el C_v es menor porque la proteína está atrapada en su estado nativo estable; hay poca libertad para fluctuar. Y, a alta temperatura ($T > 1.0$), el C_v disminuye y tiende a estabilizarse. Aquí la proteína ya está completamente desplegada (un ovillo aleatorio), y el aumento de temperatura solo incrementa la energía cinética de los residuos, sin cambiar drásticamente la estructura.

6.5 Radio de giro y grado de compactación



Se observa un aumento notorio en el R_g a partir de $T \approx 0.6$. Este cambio es la evidencia geométrica del desplegamiento.

A temperaturas altas ($T > 1.2$), el R_g alcanza una meseta en valores mucho más altos. Esto representa el estado desnaturalizado de la proteína.

A temperaturas bajas ($T < 0.5$), el R_g se mantiene en valores mínimos. Esto indica que la proteína se encuentra en su estado nativo, donde está muy plegada sobre sí misma para minimizar su energía potencial.

6.6 Comparación con el artículo base

Las gráficas generadas a partir de los datos obtenidos con HHOMD coinciden, o al menos son muy semejantes, con los resultados reportados en el artículo de Honeycutt y Thirumalai. La energía es una función continua de la temperatura. Esto indica que la transición de una estructura desplegada a una plegada corresponde a una transición de orden superior.

C_v muestra un pico en $T \simeq 0.65$. El artículo identifica este pico como el punto medio de la transición, donde la proteína pasa de un estado extendido a uno de varios posibles estados plegados compactos. El valor de $\langle R_g^2 \rangle$ se mantiene bajo (cerca de 8) a bajas temperaturas y aumenta hasta aproximadamente 24 en $T = 1.5$. Los datos de nuestras simulaciones muestran un comportamiento muy parecido: una transición de una estructura compacta a una extendida que inicia alrededor de $T \simeq 0.65$.

7 Discusión y conclusiones

En conjunto, los resultados muestran que la construcción del modelo reproduce de manera consistente el comportamiento termodinámico esperado para el plegamiento de la proteína. La transición estructural que se ve en el radio de giro coincide con el pico de calor específico en $T = 0.65$. Mientras que el C_v habla de la energía, el R_g confirma que la proteína está aumentando físicamente de tamaño (desplegándose). Ambos observables resultan herramientas complementarias para describir la transición conformacional y evaluar la validez del modelo implementado.

8 Referencias

- Anderson, J. A., Glaser, J., & Glotzer, S. C. (2020). HOOMD-blue: A Python package for high-performance molecular dynamics and hard particle Monte Carlo simulations. *Computational Materials Science*, 173, 109363. <https://doi.org/10.1016/j.commatsci.2019.109363>
- Honeycutt, J. D., & Thirumalai, D. (1992). *The nature of folded states of globular proteins*. *Biopolymers*, 32(6), 695–709.
- Muneer, A., & Mazhar, M. N. (2023). Protein folding, misfolding and related diseases. *Diseases*, 11(1), 30. <https://doi.org/10.3390/diseases11010030>